Deep Learning

Lecture 5.1: Practicum

Deep Learning in PyTorch with Tensorboard

How do I install PyTorch?

How should I handle data?

What are tensors and how do I get gradients?

How do I define models?

How do I run my models on data?

How can I compute loss?

How can I update my model parameters?

How do I save/load my models?

How do I use a GPU?

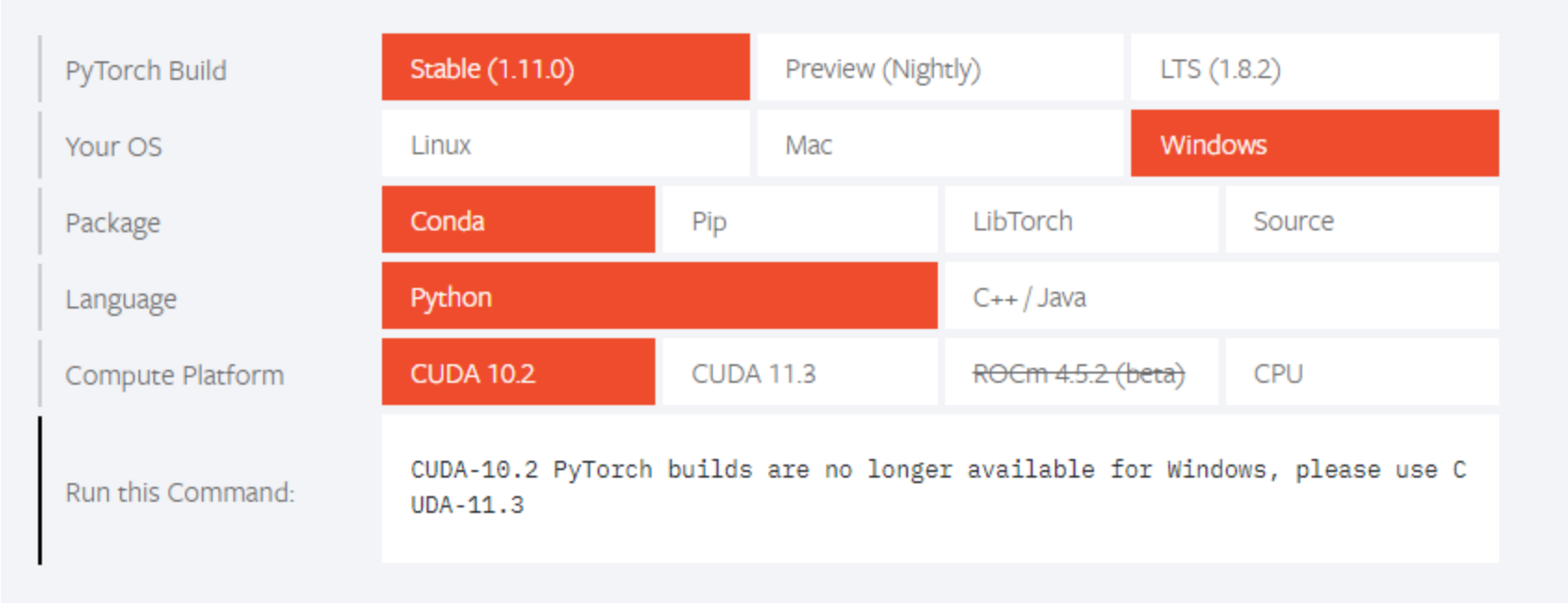
What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How do I install PyTorch?

Installation very easy with *pip* or *conda*!



https://pytorch.org/

How do I install PyTorch?

Also already installed in Google COLABs for short-run experiments:

https://colab.research.google.com/

Need-to-Know PyTorch at a Glance

	Tutorials
Define Data	https://pytorch.org/tutorials/beginner/basics/data_tutorial.html https://pytorch.org/tutorials/beginner/basics/transforms_tutorial.html
Define Model	https://pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html#sphx-glr-beginner-blitz-neural-networks-tutorial-py
Training Loop	
Get Batch of Data	https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
Produce Output from Model	https://pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html#sphx-glr-beginner-blitz-neural-networks-tutorial-py
Compute Loss	https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html https://pytorch.org/docs/stable/nn.html#loss-functions
Calculate Gradients	https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html
Update Parameters	https://pytorch.org/tutorials/beginner/basics/optimization_tutorial.html https://pytorch.org/docs/stable/optim.html
Save Model	https://pytorch.org/tutorials/beginner/basics/saveloadrun_tutorial.html

Deep Learning in PyTorch with Tensorboard

~~How do I install PyTorch?~~

How should I handle data?

What are tensors and how do I get gradients?

How do I define models?

How do I run my models on data?

How can I compute loss?

How can I update my model parameters?

How do I save/load my models?

How do I use a GPU?

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How should I handle data?

Dataset – defines what a datapoint is and how to get it from disk

- Many existing datasets already have this defined
  - Text: https://huggingface.co/docs/datasets/index
  - Vision: https://pytorch.org/vision/stable/datasets.html

```
imagenet_data = torchvision.datasets.ImageNet('path/to/imagenet_root/')
```

Dataloader – given a dataset, handles batching/shuffling, multithreaded loading automatically

```
train_dataloader = DataLoader(imagenet_data, batch_size=64, shuffle=True)
```

How should I handle data?

Dataset – defines what a datapoint is and how to get it from disk

```
class CustomDataset(Dataset):

    def __init__(self, thingsYouNeed):
      # load anything you need to load
      # to manage this dataset

    def __len__(self):
      # return how many items in the dataset


    def __getitem__(self, idx):
      # return item at index "idx"
```

Easy to make custom dataset. Just need to define how to load a single example!

Can pre-load and index list in memory if dataset size allows. Otherwise load from disk.

How should I handle data?

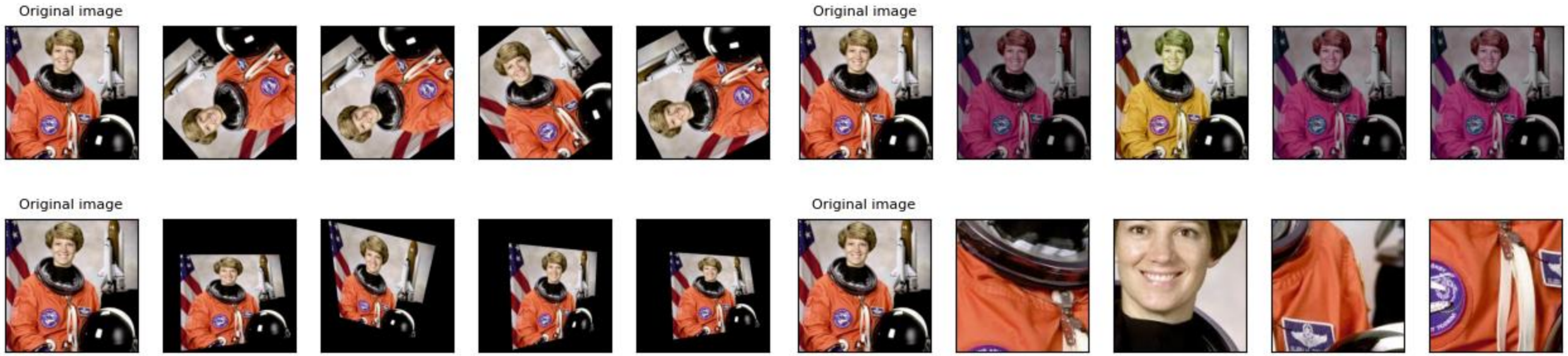
Dataset – defines what a datapoint is and how to get it from disk

```
class CustomImageDataset(Dataset):
    def __init__(self, annotations_file, img_dir, transform=None, target_transform=None):
        self.img_labels = pd.read_csv(annotations_file)
        self.img_dir = img_dir
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return len(self.img_labels)

    def __getitem__(self, idx):
        img_path = os.path.join(self.img_dir, self.img_labels.iloc[idx, 0])
        image = read_image(img_path)
        label = self.img_labels.iloc[idx, 1]
        if self.transform:
            image = self.transform(image)
        if self.target_transform:
            label = self.target_transform(label)
        return image, label
```

__getitem__ also where you can implement data augmentation

How should I handle data?

Some data augmentation already implemented in PyTorch for images:



https://pytorch.org/vision/stable/auto_examples/plot_transforms.html#sphx-glr-auto-examples-plot-transforms-py

https://pytorch.org/vision/stable/transforms.html

How should I handle data?

Dataloader – given a dataset, handles batching/shuffling, multithreaded loading automatically

```
train_dataloader = DataLoader(my_dataset, batch_size=64, shuffle=True, num_workers=4)

for x_batch, y_batch in train_dataloder:

  # do stuff!
```

Lots of options for efficiency / parallelism:

```
DataLoader(dataset, batch_size=1, shuffle=False, sampler=None,
           batch_sampler=None, num_workers=0, collate_fn=None,
           pin_memory=False, drop_last=False, timeout=0,
           worker_init_fn=None, *, prefetch_factor=2,
           persistent_workers=False)
```

https://pytorch.org/docs/stable/data.html#torch.utils.data.DataLoader

Deep Learning in PyTorch with Tensorboard

~~How do I install PyTorch?~~

~~How should I handle data?~~

What are tensors and how do I get gradients?

How do I define models?

How do I run my models on data?

How can I compute loss?

How can I update my model parameters?

How do I save/load my models?

How do I use a GPU?

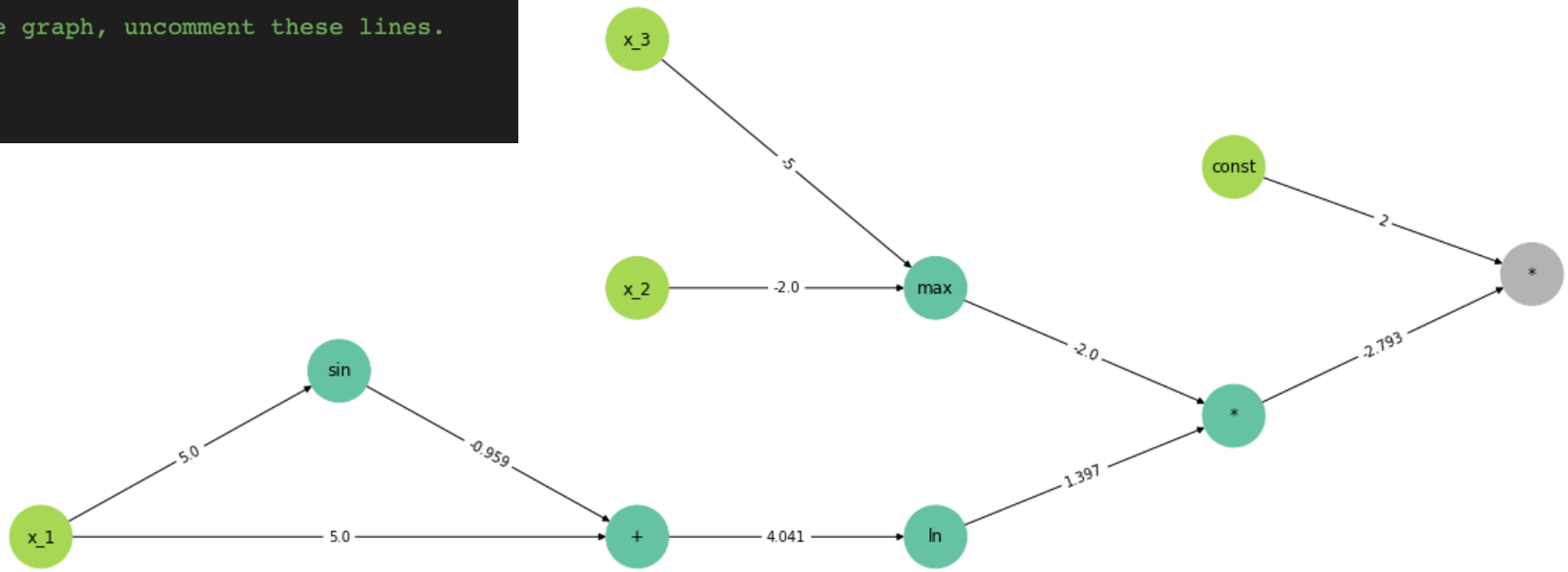
What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

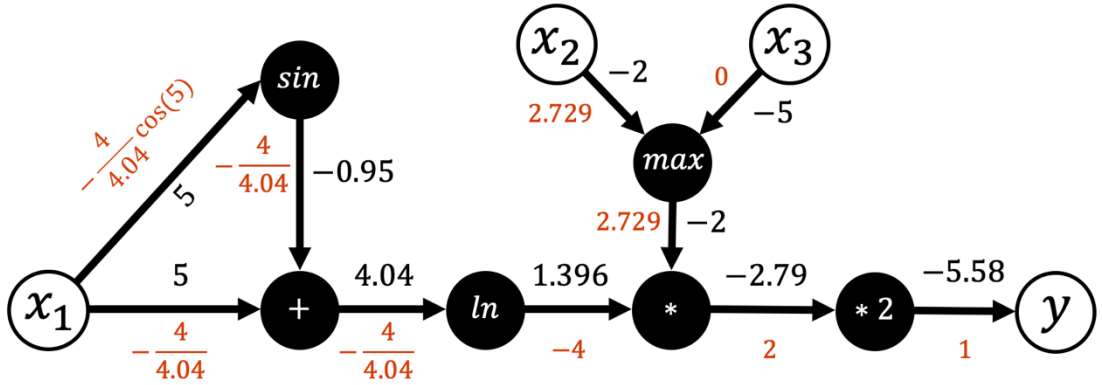
Recall Computational Graphs

```
x1 = Variable(5.0, name="x_1")
x2 = Variable(-2.0, name="x_2")
x3 = Variable(-5, name="x_3")

y = ln(x1+sin(x1))*max(x2,x3)*2

# If you want to display the graph, uncomment these lines.
plt.figure(figsize=(18,6))
y.drawGraph()
plt.show()
```



Modern Autograd Frameworks -- PyTorch

$$y = 2\log(x_1 + \sin(x_1))\max(x_2, x_3)$$



$$\frac{\delta_y}{\delta x_1} = -\frac{4}{4.04}\cos(5) - \frac{4}{4.04} \qquad \frac{\delta_y}{\delta x_2} = 2.729 \qquad \frac{\delta_y}{\delta x_3} = 0$$

```
import torch
import numpy as np

x = torch.tensor([5.0,-2.0, -5.0], requires_grad=True)
y = 2 * torch.log(x[0] + torch.sin(x[0]))*torch.max(x[1:])

y.backward()
print(x.grad)
```

```
tensor([-1.2706,  2.7930,  0.0000])
```

Modern Autograd Frameworks -- PyTorch

```
at::Tensor norm_backward(const at::Tensor & grad, const at::Tensor & self, const optional<at::Scalar> & p_, const at::Tensor & norm);
at::Tensor norm_backward(at::Tensor grad, const at::Tensor & self, const optional<at::Scalar> & p_, at::Tensor norm, at::IntArrayRef dim, bool keepdim);
at::Tensor pow_backward(at::Tensor grad, const at::Tensor & self, const at::Scalar & exponent_);
at::Tensor pow_backward_self(at::Tensor grad, const at::Tensor & self, const at::Tensor & exponent);
at::Tensor pow_backward_exponent(at::Tensor grad, const at::Tensor& self, const at::Tensor& exponent, at::Tensor result);
at::Tensor pow_backward_exponent(at::Tensor grad, const at::Scalar & base, const at::Tensor& exponent, at::Tensor result);
at::Tensor angle_backward(at::Tensor grad, const at::Tensor& self);
at::Tensor mul_tensor_backward(Tensor grad, Tensor other, ScalarType self_st);
at::Tensor div_tensor_self_backward(Tensor grad, Tensor other, ScalarType self_st);
at::Tensor div_tensor_other_backward(Tensor grad, Tensor self, Tensor other);
at::Tensor mvlgamma_backward(at::Tensor grad, const at::Tensor & self, int64_t p);
at::Tensor permute_backwards(const at::Tensor & grad, at::IntArrayRef fwd_dims);
at::Tensor rad2deg_backward(const at::Tensor& grad);
at::Tensor deg2rad_backward(const at::Tensor& grad);
at::Tensor unsqueeze_multiple(const at::Tensor & t, at::IntArrayRef dim, size_t n_dims);
at::Tensor sum_backward(const at::Tensor & grad, at::IntArrayRef sizes, at::IntArrayRef dims, bool keepdim);
at::Tensor nansum_backward(const at::Tensor & grad, const at::Tensor & self, at::IntArrayRef dims, bool keepdim);
std::vector<int64_t> reverse_list(const at::IntArrayRef list);
at::Tensor reverse_dim(const at::Tensor& t, int64_t dim);
at::Tensor prod_safe_zeros_backward(const at::Tensor &grad, const at::Tensor& inp, int64_t dim);
at::Tensor prod_backward(const at::Tensor& grad, const at::Tensor& input, const at::Tensor& result);
at::Tensor prod_backward(at::Tensor grad, const at::Tensor& input, at::Tensor result, int64_t dim, bool keepdim);
at::Tensor solve_backward_self(const at::Tensor & grad, const at::Tensor & self, const at::Tensor & A);
at::Tensor solve_backward_A(const at::Tensor & grad, const at::Tensor & self, const at::Tensor & A, const at::Tensor & solution);
at::Tensor cumsum_backward(const at::Tensor & x, int64_t dim);
at::Tensor logsumexp_backward(at::Tensor grad, const at::Tensor & self, at::Tensor result, at::IntArrayRef dim, bool keepdim);
at::Tensor logcumsumexp_backward(at::Tensor grad, const at::Tensor & self, at::Tensor result, int64_t dim);
```

https://github.com/pytorch/pytorch/blob/master/torch/csrc/autograd/FunctionsManual.h

Modern Autograd Frameworks -- PyTorch

```
Tensor sum_backward(const Tensor & grad, IntArrayRef sizes, IntArrayRef dims, bool keepdim) {
  if (!keepdim && sizes.size() > 0) {
    if (dims.size()==1) {
      return grad.unsqueeze(dims[0]).expand(sizes);
    } else {
      Tensor res = unsqueeze_multiple(grad, dims, sizes.size());
      return res.expand(sizes);
    }
  } else {
    return grad.expand(sizes);
  }
}
```

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

How do I define models?

How do I run my models on data?

How can I compute loss?

How can I update my model parameters?

How do I save/load my models?

How do I use a GPU?

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How do I define models?

Lots of neural network “layers” have already been defined in PyTorch

TORCH.NN

These are the basic building blocks for graphs:

torch.nn

- Containers
- Convolution Layers
- Pooling layers
- Padding Layers
- Non-linear Activations (weighted sum, nonlinearity)
- Non-linear Activations (other)
- Normalization Layers
- Recurrent Layers
- Transformer Layers
- Linear Layers
- Dropout Layers
- Sparse Layers
- Distance Functions
- Loss Functions
- Vision Layers
- Shuffle Layers
- DataParallel Layers (multi-GPU, distributed)
- Utilities
- Quantized Functions
- Lazy Modules Initialization

LINEAR

CLASS `torch.nn.Linear(in_features, out_features, bias=True, device=None, dtype=None)` [SOURCE]

Applies a linear transformation to the incoming data: $y = xA^T + b$

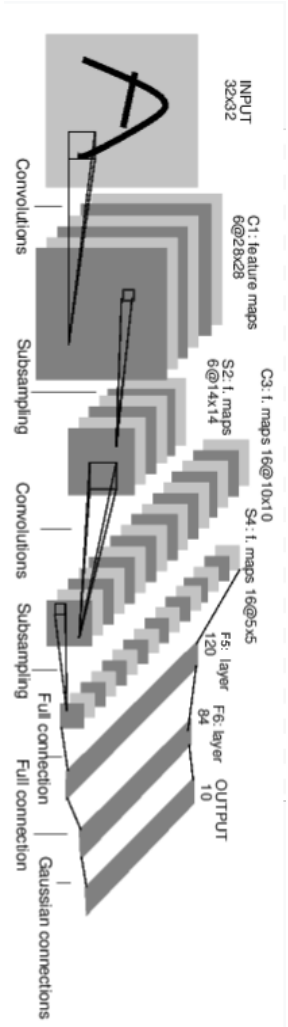
CONV2D

CLASS `torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)` [SOURCE]

https://pytorch.org/docs/stable/nn.html

How do I define models?

Can compose these together to make a model:




```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()
        # 1 input image channel, 6 output channels, 5x5 square convolution
        # kernel
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.conv2 = nn.Conv2d(6, 16, 5)
        # an affine operation: y = Wx + b
        self.fc1 = nn.Linear(16 * 5 * 5, 120)  # 5*5 from image dimension
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        # Max pooling over a (2, 2) window
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2))
        # If the size is a square, you can specify with a single number
        x = F.max_pool2d(F.relu(self.conv2(x)), 2)
        x = torch.flatten(x, 1) # flatten all dimensions except the batch dimension
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```python
input = torch.randn(1, 1, 32, 32)
out = net(input)
```

https://pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html#sphx-glr-beginner-blitz-neural-networks-tutorial-py

How do I define models?

Can also define submodules:

```python
class ResBlock(nn.Module):
    def __init__(self,inplanes: int,planes: int, stride: int = 1, padding:int = 1) -> None:
        super().__init__()

        self.conv1 = nn.Conv2d(inplanes,planes,kernel_size=3,
                               stride=stride,padding=padding,bias=False)
        self.bn1 = nn.BatchNorm2d(planes)
        self.relu1 = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(planes,planes,kernel_size=3,
                               stride=stride,padding=padding,bias=False)
        self.bn2 = nn.BatchNorm2d(planes)
        self.relu2 = nn.ReLU(inplace=True)

    def forward(self, x: Tensor) -> Tensor:
        identity = x

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu1(out)

        out = self.conv2(out)
        out = self.bn2(out)

        out += identity
        out = self.relu2(out)

        return out
```

And use them in networks:

```python
class MyResNet(nn.Module):

  def __init__(self,inplanes: int, classes: int,) -> None:
    super().__init__()
    self.conv1 = nn.Conv2d(1,128,kernel_size=3, padding=1, bias=True)
    self.block1 = ResBlock(128,128)
    self.block2 = ResBlock(128,128)
    self.conv1x1 = nn.Conv2d(128,classes,kernel_size=1, bias=True)
    self.globalpool = nn.AdaptiveMaxPool2d((1,1))

  def forward(self, x: Tensor) -> Tensor:
    out = self.conv1(x)
    out = self.block1(out)
    out = self.block2(out)
    out = self.conv1x1(out)
    out = self.globalpool(out)
    return out.squeeze()
```

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

~~How do I define models?~~

~~How do I run my models on data?~~

How can I compute loss?

How can I update my model parameters?

How do I save/load my models?

How do I use a GPU?

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How do I compute loss?

Lots of losses defined in PyTorch already. Defining new ones is easy.

Loss Functions

`nn.L1Loss`	Creates a criterion that measures the mean absolute error (MAE) between each element in the input x and target y.
`nn.MSELoss`	Creates a crit (squared L2 n target y.
`nn.CrossEntropyLoss`	This criterion input and targ
`nn.CTCLoss`	The Connecti
`nn.NLLLoss`	The negative l
`nn.PoissonNLLLoss`	Negative log l target.
`nn.GaussianNLLLoss`	Gaussian nega
`nn.KLDivLoss`	The Kullback-
`nn.BCELoss`	Creates a crit between the t
`nn.BCEWithLogitsLoss`	This loss combines a *Sigmoid* layer anc single class.
`nn.MarginRankingLoss`	Creates a criterion that measures the l $x2$, two 1D mini-batch or 0D *Tensors*, batch or 0D *Tensor* y (containing 1 or
`nn.HingeEmbeddingLoss`	Measures the loss given an input tenso tensor y (containing 1 or -1).
`nn.MultiLabelMarginLoss`	Creates a criterion that optimizes a mu classification hinge loss (margin-based x (a 2D mini-batch *Tensor*) and output *Tensor* of target class indices).
`nn.HuberLoss`	Creates a criterion that uses a squared element-wise error falls below delta ar term otherwise.
`nn.SmoothL1Loss`	Creates a criterion that uses a squared term if the absolute element-wise error falls below beta and an L1 term otherwise.
`nn.SoftMarginLoss`	Creates a criterion that optimizes a two-class classification logistic loss between input tensor x and target tensor y (containing 1 or -1).
`nn.MultiLabelSoftMarginLoss`	Creates a criterion that optimizes a multi-label one-versus-all loss based on max-entropy, between input x and target y of size (N, C).
`nn.CosineEmbeddingLoss`	Creates a criterion that measures the loss given input tensors x_1, x_2 and a *Tensor* label y with values 1 or -1.
`nn.MultiMarginLoss`	Creates a criterion that optimizes a multi-class classification hinge loss (margin-based loss) between input x (a 2D mini-batch *Tensor*) and output y (which is a 1D tensor of target class indices, $0 \leq y \leq \text{x.size}(1) - 1$):
`nn.TripletMarginLoss`	Creates a criterion that measures the triplet loss given an input tensors $x1, x2, x3$ and a margin with a value greater than 0.
`nn.TripletMarginWithDistanceLoss`	Creates a criterion that measures the triplet loss given input tensors a, p, and n (representing anchor, positive, and negative examples, respectively), and a nonnegative, real-valued function ("distance function") used to compute the relationship between the anchor and positive example ("positive distance") and the anchor and negative example ("negative distance").

https://pytorch.org/docs/stable/nn.html#loss-functions

How do I compute loss?

Build your loss somewhere:

```
criterion = torch.nn.CrossEntropyLoss()
```

Put a batch through your model and compute loss:

```
for x,y in trainloader:
  out = model(x)
  loss = criterion(out,y)
```

Compute gradients:

```
loss.backward()
```

https://pytorch.org/docs/stable/nn.html#loss-functions

How can I update my model parameters?

Many optimizers already implemented in PyTorch:

Algorithms

`Adadelta`	Implements Adadelta algorithm.
`Adagrad`	Implements Adagrad algorithm.
`Adam`	Implements Adam algorithm.
`AdamW`	Implements AdamW algorithm.
`SparseAdam`	Implements lazy version of Adam algorithm suitable for sparse tensors.
`Adamax`	Implements Adamax algorithm (a variant of Adam based on infinity norm).
`ASGD`	Implements Averaged Stochastic Gradient Descent.
`LBFGS`	Implements L-BFGS algorithm, heavily inspired by minFunc.
`NAdam`	Implements NAdam algorithm.
`RAdam`	Implements RAdam algorithm.
`RMSprop`	Implements RMSprop algorithm.
`Rprop`	Implements the resilient backpropagation algorithm.
`SGD`	Implements stochastic gradient descent (optionally with momentum).

https://pytorch.org/docs/stable/optim.html#

How can I update my model parameters?

1) Register parameters with the optimizer and set hyperparameter
2) Reset the gradient
3) Compute loss function
4) Run backward
5) Run step from the optimizer to update loss

```
>>> optimizer = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
>>> optimizer.zero_grad()
>>> loss_fn(model(input), target).backward()
>>> optimizer.step()
```

https://pytorch.org/docs/stable/optim.html#

Deep Learning in PyTorch with Tensorboard

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

~~How do I define models?~~

~~How do I run my models on data?~~

~~How can I compute loss?~~

~~How can I update my model parameters?~~

How do I save/load my models?

How do I use a GPU?

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How can I update my model parameters?

Saving model checkpoints:

```
# Additional information
EPOCH = 5
PATH = "model.pt"
LOSS = 0.4

torch.save({
            'epoch': EPOCH,
            'model_state_dict': net.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'loss': LOSS,
            }, PATH)
```

Loading model checkpoints:

```
model = Net()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)

checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
epoch = checkpoint['epoch']
loss = checkpoint['loss']

model.eval()
# - or -
model.train()
```

https://pytorch.org/tutorials/recipes/recipes/saving_and_loading_a_general_checkpoint.html

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

~~How do I define models?~~

~~How do I run my models on data?~~

~~How can I compute loss?~~

~~How can I update my model parameters?~~

~~How do I save/load my models?~~

~~How do I use a GPU?~~

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

How to use GPUs?

Detect GPU and move model parameters to it:

```
device = torch.device('cuda')

model.to(device)
```

Also need to move data to the GPU:

```
for x,y in trainloader:
  x = x.to(device)
  y = y.to(device)
  out = model(x)
```

https://pytorch.org/tutorials/recipes/recipes/saving_and_loading_a_general_checkpoint.html

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

~~How do I define models?~~

~~How do I run my models on data?~~

~~How can I compute loss?~~

~~How can I update my model parameters?~~

~~How do I save/load my models?~~

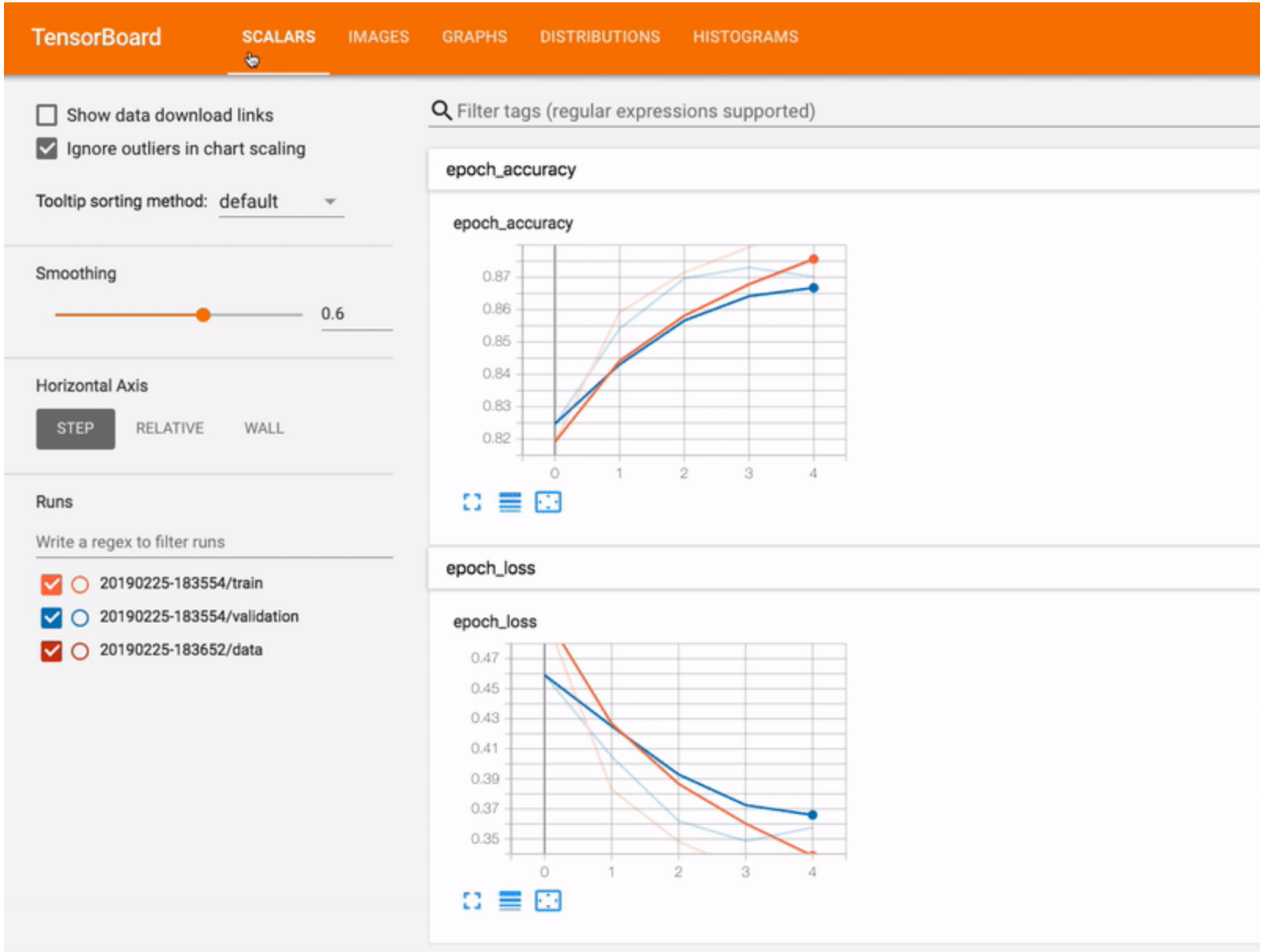
~~How do I use a GPU?~~

What is Tensorboard?

How can I monitor training?

What can I do beyond just showing loss / accuracy plots?

What is Tensorboard?



https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html

How can I monitor training?

Create a Tensorboard writer:

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter("logs/mymodel", comment="my model 4/24")
```

Write things to it every iteration or epoch:

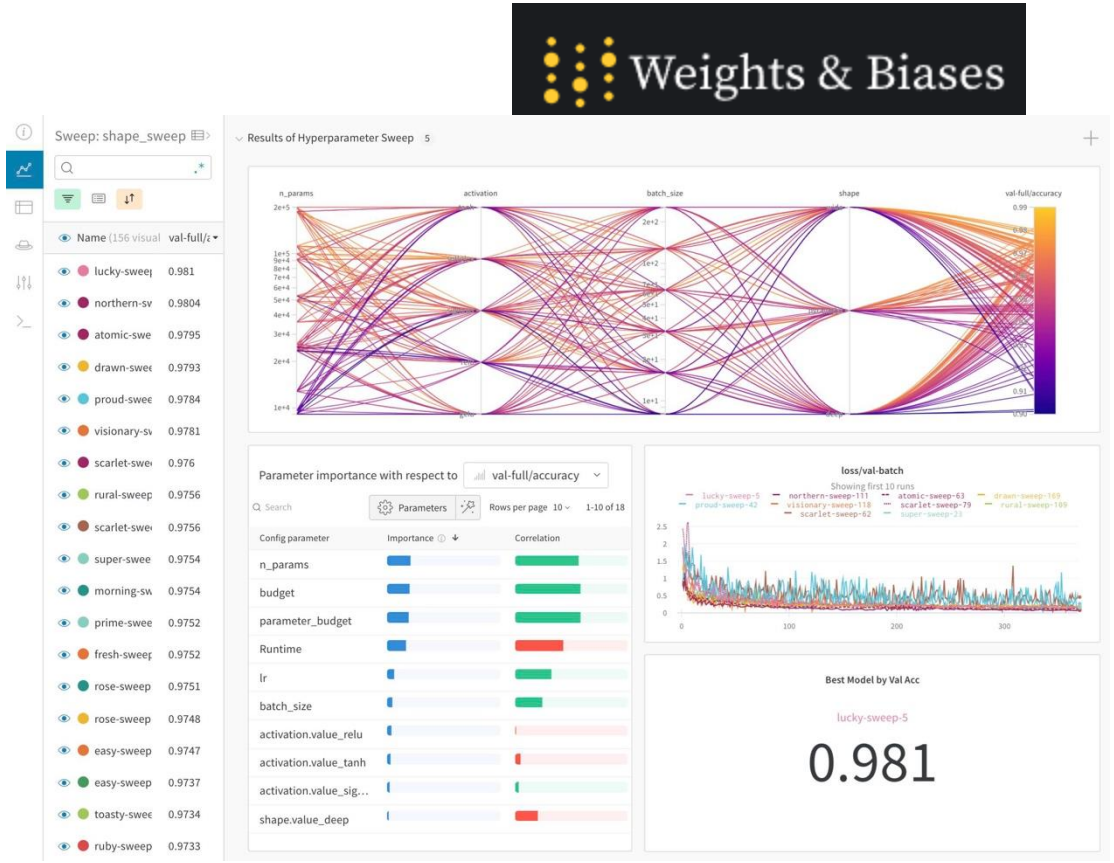
```
writer.add_scalar('Loss/train', loss.item(), iter)
writer.add_scalar('Acc/train', correct, iter)
```

https://pytorch.org/tutorials/recipes/recipes/saving_and_loading_a_general_checkpoint.html

Beyond Scalar Logging

https://pytorch.org/tutorials/intermediate/tensorboard_tutorial.html

Worth Noting:



A lot of people prefer this now – better for tracking lots of experiments

Deep Learning in PyTorch with Tensorboard

~~How do I install PyTorch?~~

~~How should I handle data?~~

~~What are tensors and how do I get gradients?~~

~~How do I define models?~~

~~How do I run my models on data?~~

~~How can I compute loss?~~

~~How can I update my model parameters?~~

~~How do I save/load my models?~~

~~How do I use a GPU?~~

~~What is Tensorboard?~~

~~How can I monitor training?~~

~~What can I do beyond just showing loss / accuracy plots?~~

Demo

https://colab.research.google.com/drive/1C99BE2uR_9fREWaWdGkKug_ArRpIlAWJ?usp=sharing